



VIT[®]

BHOPAL

PIYUSH KUMAR SINGH

25BAI11043

COMPUTER SCIENCE AND ENGINEERING (AIML)

2025-2029 BATCH

JAVA PROJECT REPORT

STUDENT PERFORMANCE MANAGEMENT SYSTEM

18TH SEPTEMBER 2026

- *Abstract*

The Student Performance Management System is an application written in Java which is designed for the efficient management of student records and academic performance. The main goal of the project is to remove the problems that come with managing student information manually. It also aims to make it easy to get access, to academic data whenever needed.

Users can add, modify, delete, and search for student records. The system is also capable of entering marks, working out percentages and grades, producing individual performance reports, identifying the students with the highest and lowest scores, computing the average score for each class, and carrying out a pass/fail analysis. The student data is stored in a CSV file, which means that the records can easily be saved and retrieved.

The project makes use of fundamental Java features such as Object-Oriented Programming, the Collections Framework, File Handling, and Exception Handling. It employs custom exceptions in order to deal with invalid marks and missing student records, which makes the system more reliable and user-friendly.

The project contributed to a better understanding of how Java concepts can be put to use when solving real-world problems and also helped to improve one's abilities in areas such as software design, data management, error handling, and project development. The system provides an useful way to track student progress. It can be improved in the future by adding displays and support, for storing data in a database.

● Introduction

Managing student records is a part of every institution. When student records are kept by hand student records become hard to watch student details, marks, grades and overall performance. Manual keeping of student records can also lead to calculation mistakes, data loss and more effort when searching or updating information.

The Student Performance Management System is a Java based console application that helps manage student information in a way. I think the system is very useful for keeping track of student details. The Student Performance Management System makes it easy to handle student data. The system lets users add students update existing student records, delete student records, search for students and enter marks for various subjects.

In addition to student record management the system can automatically calculate percentages and grades generate individual student reports spot the lowest scorers find class averages and do pass/fail analysis for student data. Student data is saved in a CSV file so that student information is not lost when the program closes.

The project was built using Java concepts such as classes and objects, collections, file handling and exception handling. Special exceptions are used to handle situations, like marks or searching for a student who does not exist in the student records.

Student record management is the aim of this project. Student record management should become easier and more accurate. The project also gives experience in using Java programming concepts to solve a real-world student records problem. The project shows how software can automate tasks and improve efficiency.

● Objectives

The major objectives of the project are:

1. To build a system for keeping and handling student information.
2. To let people add, change, remove and look up student records.
3. To offer a method, for entering and managing students scores.
4. To automatically calculate the percentage and grade of a student.
5. To generate individual performance reports.
6. To calculate the class average and identify the highest and lowest scorers.
7. To perform pass/fail analysis based on students' marks.
8. To store student information in a CSV file so that the data can be saved and used later.
9. To use exception handling for situations such as invalid marks or a student record not being found.
10. To apply Java concepts such as OOP, collections, file handling, and exception handling in a practical project.

● Problem Statement

Manually keeping track of student marks, percentages, and grades in a classroom or small institute can be cumbersome and prone to mistakes if not done using specialized tools like pen-and-paper method or basic spreadsheet tools not tailored for this task: calculating percentages manually, forgetting whom which percentage belongs to, comparing students to As the class grows, the average turns out to be quite boring. There arises the need for a small, lightweight program without external dependencies that a teacher or student coordinator could run immediately without any setup of database or web server for managing student data.

- *Functional Requirements*

The main functions of the Student Performance Management System are:

- *Student Management*

- * Add a new student with their basic details.
- * Update the details of an existing student.
- * Delete a student record when required.
- * Search for a student using their details.

- *Marks Management*

- * Enter marks for different subjects.
- * Check whether the entered marks are valid.
- * Store the marks with the student's record.
- * Display the marks of a student when needed.

- *Percentage and Grade*

- * Calculate the total marks obtained by a student.
- * Calculate the student's percentage.
- * Assign a grade according to the percentage.
- * Show whether the student has passed or failed.

- *Performance Report*

- * Display the performance details of an individual student.
- * Calculate the average marks of the class.
- * Find the student with the highest marks.
- * Find the student with the lowest marks.
- * Show the pass and fail details of the class.

- *File Handling*

- * Load student records, from the CSV file when the program starts.
- * Save student records to the CSV file after making changes so that student records stay current.
- * Keep student records saved so that student records can be accessed the time the program runs.

- *File Handling*

- * Show a message if a student cannot be found.
- * Prevent incorrect data from being added to the records

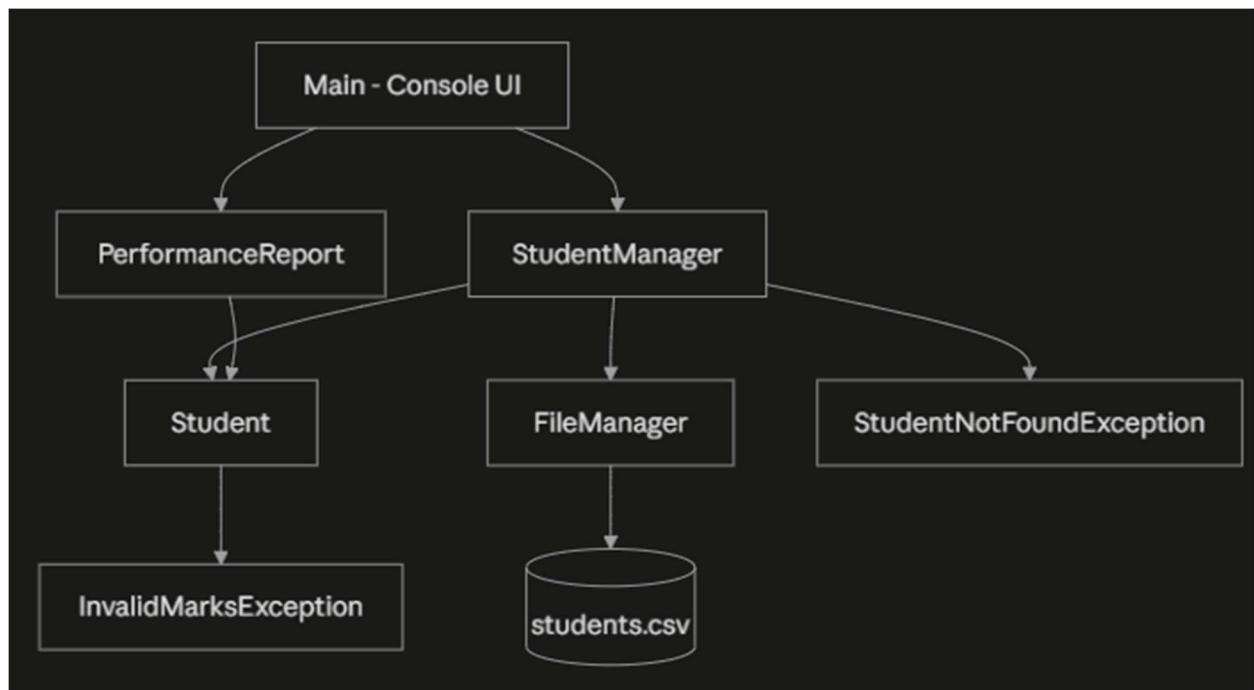
- *Non-Functional Requirements*

1. **Performance** : CRUD operations and report generation complete in under 1 second for a roster of a few hundred students, since everything is held in memory as an `ArrayList<Student>` .
2. **Reliability** : Every add, update, delete, or marks entry is immediately persisted to `students.csv` , so no data is lost if the program exits unexpectedly.
3. **Usability** : A numbered console menu (0–10) guides the user through every operation with clear prompts, so no prior documentation is needed to operate the system.
4. **Maintainability** : Each responsibility is isolated into its own class (`Student` , `StudentManager` , `FileManager` , `PerformanceReport`), so a change to one concern (e.g. storage format) doesn't require touching the others.
5. **Error Handling / Robustness** : Custom checked exceptions (`InvalidMarksException` , `StudentNotFoundException`) force calling code to handle bad input explicitly rather than failing silently; a corrupted CSV line is skipped with a warning instead of crashing the whole load.
6. **Resource Efficiency** : No external dependencies, no database server, and no build tool overhead, the entire system runs on a bare JDK install with a CSV file a few bytes in size.

• System Architecture

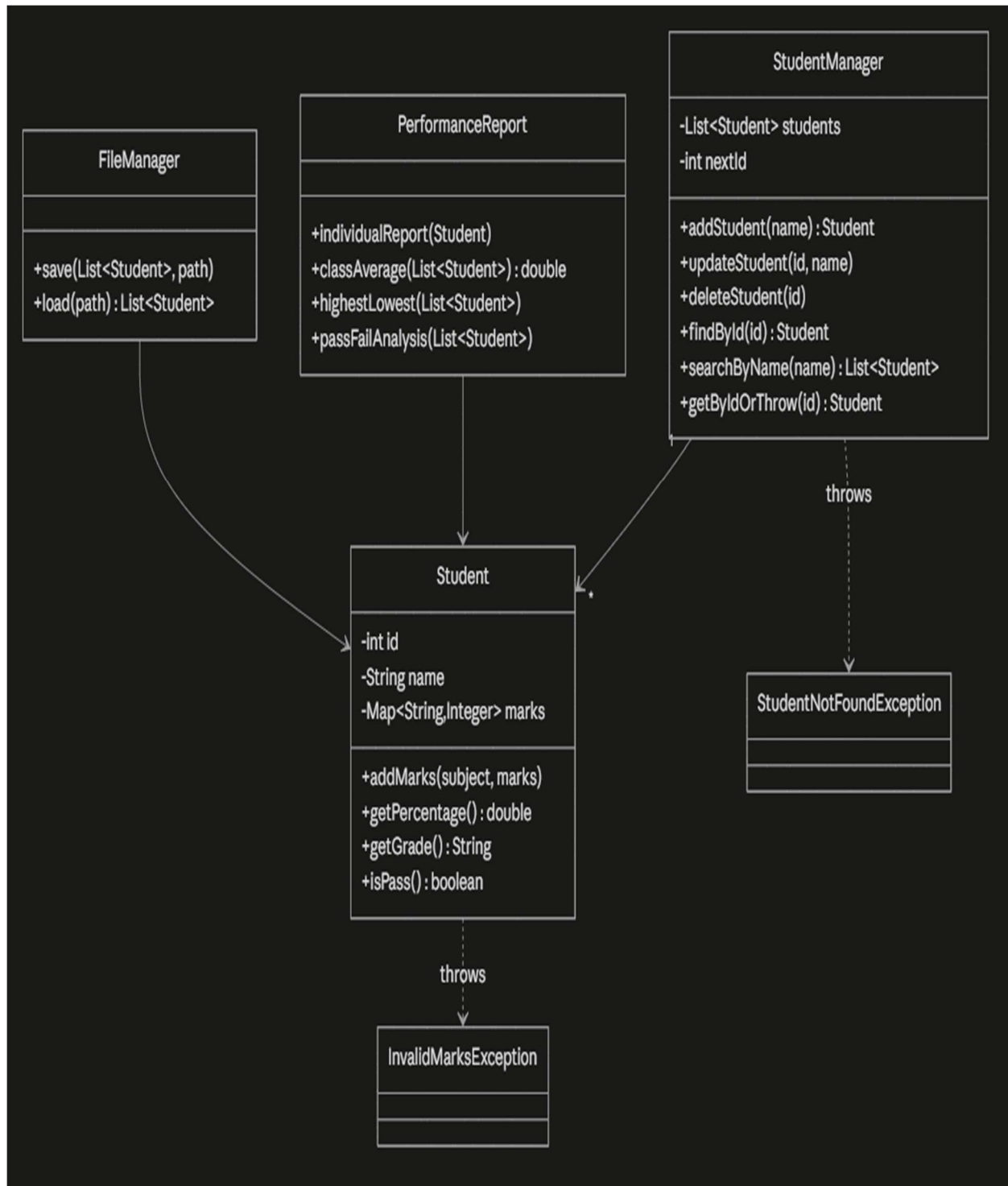
The system is divided into different modules:

- Main Console – Provides the menu and takes user input.
- Student – Stores student details and marks.
- Student Manager – Handles add, update, delete, and search operations.
- File Manager – Saves and loads data from the CSV file.
- Performance Report – Calculates percentage, grade, average, and performance.
- Exception Handling – Handles invalid marks and missing student records.

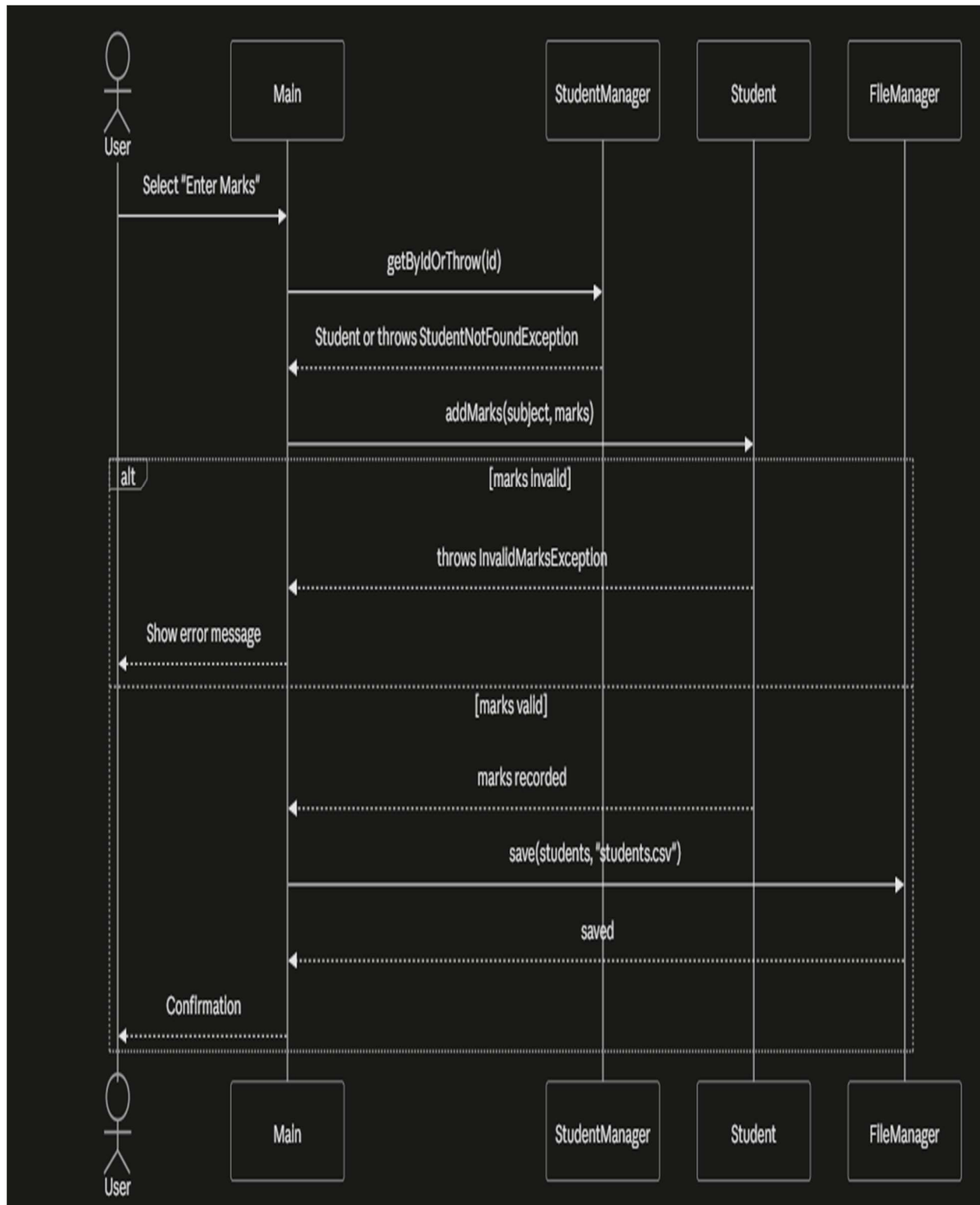


• Design Diagram

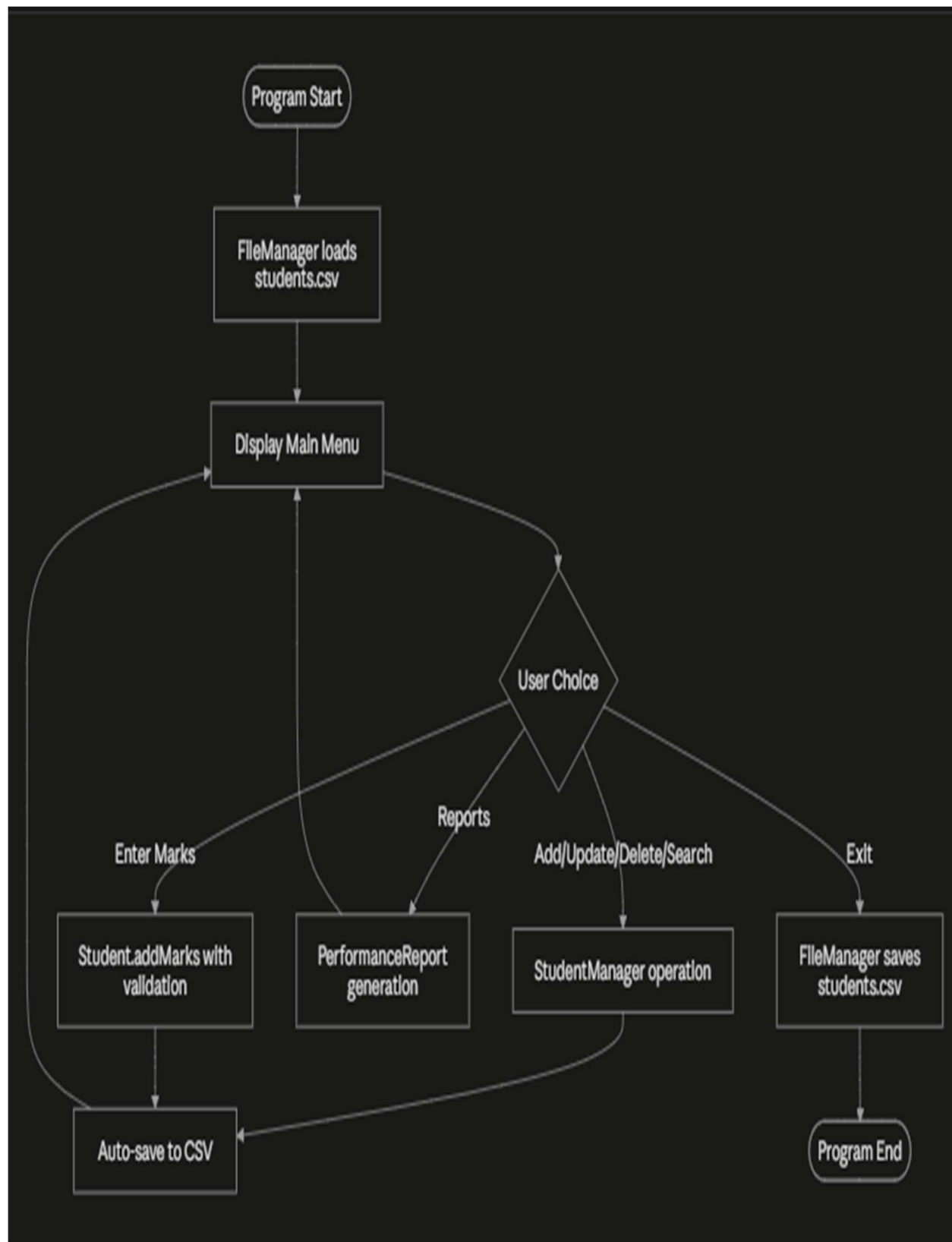
Class diagram:



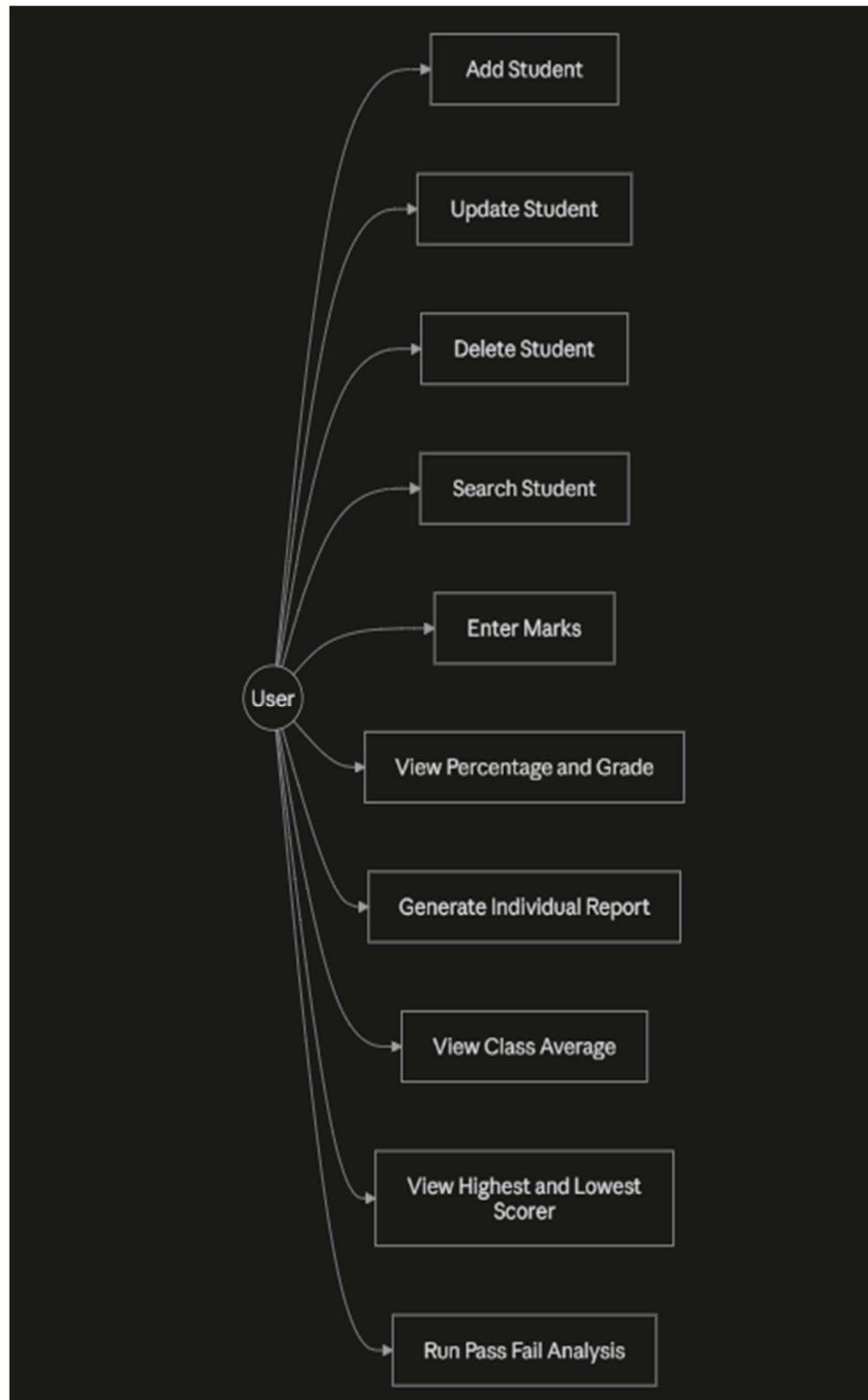
Sequence diagram:



Workflow diagram:



Usecase diagram:



• Database Storage Design

This project does not use a relational database, so no ER diagram applies. Storage is a flat CSV file, one line per student, in the format:

1	1	Piyush	English:96	Maths:100	Hindi:97
2	2	Ayush	Maths:90	English:86	Hindi:92
3	3	Arghav	Maths:98	English:96	Hindi:90

FileManager.java is the only class that reads or writes this format, so switching to a real database later would only require rewriting that one class.

• Design Decisions & Rationale

- Layered design over one big class: splitting Student / StudentManager / PerformanceReport / FileManager / Main means each class has one reason to change, and the logic is easy to unit-test in isolation (see AppTest.java).
- Map for marks instead of fixed fields: lets a student have any number of subjects without hard-coding 'subject1, subject2, ...' fields.
- Custom checked exceptions instead of boolean return values: forces the caller (Main) to explicitly decide what happens on bad input rather than silently ignoring a 'false' return, which is easy to forget.
- Plain CSV file instead of a database: the assignment's scope (a single class/section, run locally) does not justify the added complexity of a DB

engine or ORM; CSV is human-readable and trivial to inspect while debugging.

- Auto-save after every mutating action: guarantees the file on disk is never more than one action out of date, protecting against data loss if the program is closed unexpectedly.

● Implementation Details

The project has 8 Java source files under src/:

File	Responsibility
Student.java	Model: one student's ID, name, marks; calculates total, percentage, grade
StudentManager.java	Owns the ArrayList<Student> roster; add/update/delete/search
PerformanceReport.java	Static report methods: individual report, class average, highest/lowest, pass/fail
FileManager.java	Saves/loads the roster to/from students.csv using Java File I/O
InvalidMarksException.java	Custom checked exception for out-of-range or blank marks input
StudentNotFoundException.java	Custom checked exception for lookups on a non-existent ID
Main.java	Console menu loop; wires the other classes together; central exception handling
AppTest.java	Dependency-free test harness covering all core logic

Key algorithm:

- Grading scale: ≥ 90 A+, ≥ 80 A, ≥ 70 B, ≥ 60 C, ≥ 50 D, ≥ 40 E, else F
- Pass/Fail rule: a student passes if overall percentage $\geq 40\%$
- Highest/Lowest: single linear scan over the roster comparing `getPercentage()`
- ID assignment: an auto-incrementing counter in `StudentManager`, kept ahead of any ID restored from disk so new and reloaded IDs never collide

Validation And Handling Error:

`Student.addMarks()` rejects marks outside 0-100 and blank subject names by throwing `InvalidMarksException`. `StudentManager.getByIdOrThrow()` throws `StudentNotFoundException` for any ID that doesn't exist. `Main.java` catches both exception types in one place around the menu dispatch, so every operation fails safely with a clear message instead of crashing or corrupting the in-memory roster.

- *Result's*

Add Student:

```
PS C:\Users\piyus\OneDrive\Desktop\Vityarthi_Java_Project_25BAI11043>  
C:\Users\piyus\AppData\Roaming\Code\User\workspaceStorage\bec93fbacaf2
```

```
===== STUDENT PERFORMANCE MANAGEMENT SYSTEM =====
```

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Enter Marks
6. View Percentage & Grade
7. Individual Report
8. Class Average
9. Highest/Lowest Marks
10. Pass/Fail Analysis
0. Save & Exit

```
Enter your choice: 1
```

```
Enter student name: Piyush
```

```
Student added successfully! Assigned ID = 1
```

Class Average:

```
PS C:\Users\piyus\OneDrive\Desktop\Vityarthi_Java_Project_25BAI11043>  
C:\Users\piyus\AppData\Roaming\Code\User\workspaceStorage\bec93fbacaf2  
Loaded 3 student(s) from students.csv
```

```
===== STUDENT PERFORMANCE MANAGEMENT SYSTEM =====
```

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Enter Marks
6. View Percentage & Grade
7. Individual Report
8. Class Average
9. Highest/Lowest Marks
10. Pass/Fail Analysis
0. Save & Exit

```
Enter your choice: 8
```

```
Class Average Percentage: 93.89% (based on 3 students)
```

Delete Student:

```
PS C:\Users\piyus\OneDrive\Desktop\Vityarthi_Java_Project_25BAI11043>
Messages' '-cp' 'C:\Users\piyus\AppData\Roaming\Code\User\workspaceSto
Main'
Loaded 1 student(s) from students.csv

===== STUDENT PERFORMANCE MANAGEMENT SYSTEM =====
1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Enter Marks
6. View Percentage & Grade
7. Individual Report
8. Class Average
9. Highest/Lowest Marks
10. Pass/Fail Analysis
0. Save & Exit
Enter your choice: 3
Enter student ID to delete: 1
Student deleted successfully!
```

Enter Marks:

```
PS C:\Users\piyus\OneDrive\Desktop\Vityarthi_Java_Project_25BAI11043>  
C:\Users\piyus\AppData\Roaming\Code\User\workspaceStorage\bec93fbacaf2  
Loaded 3 student(s) from students.csv
```

```
===== STUDENT PERFORMANCE MANAGEMENT SYSTEM =====
```

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Enter Marks
6. View Percentage & Grade
7. Individual Report
8. Class Average
9. Highest/Lowest Marks
10. Pass/Fail Analysis
0. Save & Exit

```
Enter your choice: 5
```

```
Enter student ID: 3
```

```
Enter subject name: Maths
```

```
Enter marks obtained (0-100): 98
```

```
Marks recorded for Arghav in Maths
```

Highest lowest:

```
PS C:\Users\piyus\OneDrive\Desktop\Vityarthi_Java_Project_25BAI11043> & 'C:\Program Files\Java\jdk-9.0.4\bin\java.exe' -Xmx1024m -Xms128m -Duser.timezone=Asia/Kolkata -Duser.language=en -Duser.country=IN -Duser.variant= -Duser.home=C:\Users\piyus\AppData\Roaming\Code\User\workspaceStorage\bec93fbacaf25cf68798862f0 Loaded 3 student(s) from students.csv
```

```
===== STUDENT PERFORMANCE MANAGEMENT SYSTEM =====
```

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Enter Marks
6. View Percentage & Grade
7. Individual Report
8. Class Average
9. Highest/Lowest Marks
10. Pass/Fail Analysis
0. Save & Exit

Enter your choice: 9

Highest Scorer: ID: 1	Name: Piyush	Percentage: 97.67%	Grade: A+
Lowest Scorer : ID: 2	Name: Ayush	Percentage: 89.33%	Grade: A

Individual report:

```
PS C:\Users\piyus\OneDrive\Desktop\Vityarthi_Java_Project_25BAI11043>
Messages' '-cp' 'C:\Users\piyus\AppData\Roaming\Code\User\workspaceSto
Main'
Loaded 3 student(s) from students.csv
```

```
===== STUDENT PERFORMANCE MANAGEMENT SYSTEM =====
```

```
1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Enter Marks
6. View Percentage & Grade
7. Individual Report
8. Class Average
9. Highest/Lowest Marks
10. Pass/Fail Analysis
0. Save & Exit
Enter your choice: 7
Enter student ID: 2
```

```
----- Individual Report -----
```

```
ID : 2
```

```
Name : Ayush
```

```
Subject-wise marks:
```

```
Maths : 90/100
```

```
English : 86/100
```

```
Hindi : 92/100
```

```
Total : 268 / 300
```

```
Percentage : 89.33%
```

```
Grade : A
```

```
Result : PASS
```

```
-----
```

Main menu:

```
PS C:\Users\piyus\OneDrive\Desktop\Vityarthi_Java_Project_25BAI11043>  
C:\Users\piyus\AppData\Roaming\Code\User\workspaceStorage\bec93fbacaf2
```

```
===== STUDENT PERFORMANCE MANAGEMENT SYSTEM =====
```

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Enter Marks
6. View Percentage & Grade
7. Individual Report
8. Class Average
9. Highest/Lowest Marks
10. Pass/Fail Analysis
0. Save & Exit

```
Enter your choice:
```

Pass fail:

```
PS C:\Users\piyus\OneDrive\Desktop\Vityarthi_Java_Project_25BAI11043>
C:\Users\piyus\AppData\Roaming\Code\User\workspaceStorage\bec93fbacaf2
Loaded 3 student(s) from students.csv
```

```
===== STUDENT PERFORMANCE MANAGEMENT SYSTEM =====
```

1. Add Student
 2. Update Student
 3. Delete Student
 4. Search Student
 5. Enter Marks
 6. View Percentage & Grade
 7. Individual Report
 8. Class Average
 9. Highest/Lowest Marks
 10. Pass/Fail Analysis
 0. Save & Exit
- Enter your choice: 10

```
----- Pass/Fail Analysis -----
```

```
Total Students : 3
Passed          : 3
Failed          : 0
Pass Rate       : 100.00%
```

Percentage Grade:

```
PS C:\Users\piyus\OneDrive\Desktop\Vityarthi_Java_Project_25BAI11043>  
C:\Users\piyus\AppData\Roaming\Code\User\workspaceStorage\bec93fbacaf29  
Loaded 3 student(s) from students.csv
```

```
===== STUDENT PERFORMANCE MANAGEMENT SYSTEM =====
```

1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Enter Marks
6. View Percentage & Grade
7. Individual Report
8. Class Average
9. Highest/Lowest Marks
10. Pass/Fail Analysis
0. Save & Exit

```
Enter your choice: 6
```

```
Enter student ID: 3
```

```
Percentage: 94.67% Grade: A+
```

Save load:

```
PS C:\Users\piyus\OneDrive\Desktop\Vityarthi_Java_Project_25BAI11043>
● C:\Users\piyus\AppData\Roaming\Code\User\workspaceStorage\bec93fbacaf2
Loaded 3 student(s) from students.csv

===== STUDENT PERFORMANCE MANAGEMENT SYSTEM =====
1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Enter Marks
6. View Percentage & Grade
7. Individual Report
8. Class Average
9. Highest/Lowest Marks
10. Pass/Fail Analysis
0. Save & Exit
Enter your choice: 0
Data saved. Exiting... Goodbye!
```

Search student:

```
PS C:\Users\piyus\OneDrive\Desktop\Vityarthi_Java_Project_25BAI11043>
Messages' '-cp' 'C:\Users\piyus\AppData\Roaming\Code\User\workspaceStor
Main'
Loaded 3 student(s) from students.csv

===== STUDENT PERFORMANCE MANAGEMENT SYSTEM =====
1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Enter Marks
6. View Percentage & Grade
7. Individual Report
8. Class Average
9. Highest/Lowest Marks
10. Pass/Fail Analysis
    0. Save & Exit
Enter your choice: 4
Search by: 1) ID  2) Name
Choice: 1
Enter ID: 1
ID: 1    Name: Piyush           Percentage: 97.67% Grade: A+
```

Test run:

```
PS C:\Users\piyus\OneDrive\Desktop\Vityarthi_Java_Project_25BAI11043>
C:\Users\piyus\AppData\Roaming\Code\User\workspaceStorage\bec93fbacaf2
[PASS] addStudent assigns ID 1 to first student
[PASS] findById locates the newly added student
[PASS] updateStudent changes the name
[PASS] deleteStudent removes the student
[PASS] updateStudent throws StudentNotFoundException for a bad ID
[PASS] Percentage calculated correctly (95+85)/200
[PASS] Grade for 90% is A+
[PASS] isPass() is true for 90%
[PASS] Marks > 100 are rejected
[PASS] Negative marks are rejected
[PASS] Class average is (80+20)/2 = 50%
[PASS] Student A passes
[PASS] Student B fails
[PASS] Reloaded manager has 1 student
[PASS] Reloaded student has correct name
[PASS] Reloaded student has correct marks

===== TEST SUMMARY =====
Passed: 16   Failed: 0
```

Update student:

```
PS C:\Users\piyus\OneDrive\Desktop\Vityarthi_Java_Project_25BAI11043>
Messages' '-cp' 'C:\Users\piyus\AppData\Roaming\Code\User\workspaceSto
Main'
Loaded 1 student(s) from students.csv

===== STUDENT PERFORMANCE MANAGEMENT SYSTEM =====
1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. Enter Marks
6. View Percentage & Grade
7. Individual Report
8. Class Average
9. Highest/Lowest Marks
10. Pass/Fail Analysis
0. Save & Exit
Enter your choice: 2
Enter student ID to update: 1
Enter new name: Ayush
Student updated successfully!
```

• Testing Approach

Testing is done with a small, dependency-free harness (AppTest.java) rather than an external framework like JUnit, since the assignment scope did not call for build-tool dependencies. Each test method exercises one behaviour and prints a PASS/FAIL verdict; a summary line reports the totals.

Test	What it verifies
testAddAndFindStudent	New students get sequential IDs and can be found by ID
testUpdateStudent	Updating a name persists on the same object
testDeleteStudent	Deleted students can no longer be found
testUpdateNonExistentStudentThrows	StudentNotFoundException is thrown for a bad ID
testGradeCalculation	Percentage and grade are computed correctly from marks
testInvalidMarksRejected	Marks above 100 or negative are rejected with InvalidMarksException
testClassAverageAndPassFail	Class average and individual pass/fail results are correct
testFilePersistence	A save() followed by load() round-trips a student's data exactly

Result on the current codebase: all 16 assertions pass (java -cp bin AppTest).

• Challenges Faced

- Deciding between returning boolean flags versus throwing custom exceptions for failure cases - exceptions were chosen because they cannot be silently ignored the way an unchecked boolean return can.
- Keeping the auto-incrementing ID counter consistent after reloading data from disk, so a freshly added student can never accidentally reuse an ID that was restored from a previous session.
- Designing the CSV format to support a variable number of subjects per student without needing a more complex format - solved with a nested delimiter scheme (comma for fields, semicolon between subjects, colon between subject and mark).

/

• Learning & Key Takeaways

- Custom checked exceptions make error paths visible in the code (via throws clauses) instead of hidden inside 'if (!ok)' checks scattered through the caller.
- Separating a program into model / manager / report / persistence / UI layers makes each piece independently testable, which is what made the AppTest harness straightforward to write.
- A simple plain-text file format, designed deliberately, can support fairly rich data (a variable number of subjects per student) without needing a database.

● *Future Enhancement*

- Support multiple classes/sections instead of a single flat roster
- Replace the CSV file with a real database (e.g. SQLite via JDBC) for larger datasets
- Add a graphical (JavaFX/Swing) or web-based front end in place of the console menu
- Add authentication so students and teachers see different views/permissions
- Export individual and class reports to PDF/Excel

● *Reference*

- Oracle Java SE 17 Documentation :
docs.oracle.com/en/java/javase/17/
- Java Collections Framework guide :
docs.oracle.com/javase/tutorial/collections/
- Java Exceptions tutorial :
docs.oracle.com/javase/tutorial/essential/exceptions/
- VITYarthi 'Build Your Own Project' guidelines document (course-provided)
- Project GitHub Repository Vityarthi Java Project
- Java File Handling Documentation

